Lab 2: Network Layer

Userspace IPv4 and Distance-Vector Routing

October 27, 2026

Implementation and source arrangement

This lab extends the userspace Ethernet library with IPv4 transmission, local delivery, multi-hop forwarding, and dynamic routing. The routing metric is hop count. The implementation and CP3-CP6 experiments were verified on `lab-comp-net`, using Linux 6.8.0, GCC 13.3.0, and libpcap 1.10.4. The experiments were performed on October 27, 2026.

The listing uses the submission layout: evaluation programs are under `src/test/`, with the required vnetUtils scripts under `src/test/vnetUtils/helper/`. The original development tree and recorded commands use `test/` and `vnetUtils/helper/`; the submitted build and experiment runners use the relocated paths.

Source	Responsibility
`src/device.c`, `device.h`	Register interfaces; cache their MAC and IPv4 addresses; own separate RX and TX handles.
`src/packetio.c`, `packetio.h`	Construct and capture Ethernet II frames; dispatch the frame callback; serialize frame injection.
`src/ip.c`, `ip.h`	PT3: construct, validate, deliver, and forward IPv4 datagrams; register the IP callback.
`src/smtroute.c`, `smtroute.h`	PT3: manual routes, longest-prefix lookup, DV messages, local routes, timeout, and garbage collection.
`src/smtarp.c`, `smtarp.h`	ARP request/reply construction and processing; synchronized ARP cache.
`src/main.c`	Register interfaces and local routes, dispatch frames, start receive threads, and periodically maintain and advertise routes.

Checkpoint entrypoints are `src/test/cp3/send_ipv4_demo.c` (known-MAC transmission), `src/test/cp4/router.c` (interactive routing and sending for CP4/CP5), `src/test/cp5/run.py` (six-node experiment), and `src/test/cp6/lpm.c` with `run.py` (LPM and two-outlet capture). `src/test/checkpoint_common.py` supplies shared topology, interaction, capture, verification, and cleanup tools.

From the repository root, build the main program and checkpoint programs with:

```
make all cp3 cp4 cp5 cp6
```

The build uses `-O2 -Wall -Wextra -Wpedantic -Werror -pthread` and links libpcap. Executables are placed in `tmp/`. `make cp5` reuses `tmp/cp4-router`. The main program accepts one or more interface names; the checkpoint router additionally supports `route <ip>`, `send <ip> <payload>`, and `quit`.

WT2 - Packet trace analysis

Question	Answer
Which ARP Reply field equals the Sender MAC in the corresponding ARP Request?	The **Target MAC address** (target hardware address).
How many IPv4 packets have DF unset?	**6**.
Base IPv4 and IPv6 header lengths, excluding options/extensions?	**20 bytes** for IPv4 and **40 bytes** for IPv6.

The DF count was independently checked against `pcap-trace/trace.pcap`. The matching frame numbers are 12, 17, 18, 32, 38, and 44; the corresponding Wireshark display filter is `ip.flags.df == 0`. In the ARP exchange at frames 33 and 34, the Request Sender MAC and Reply Target MAC both equal `6a:15:0a:ba:9b:7c`.

WT3 - Determining the next-hop MAC address

The application supplies source/destination IP addresses, protocol, and payload to `sendIPPacket()`. It does not supply an Ethernet destination. The IP layer calls `findRouter()` with the destination IP, selects the longest matching valid route, and obtains the outgoing interface and next-hop MAC from that route.

The route obtains its MAC in either of the following ways:

1. **Manual routing:** `setRoutingTable()` explicitly receives a next-hop MAC and interface. CP3 uses this interface to configure a host route before sending.
2. **Dynamic routing:** `handleDV()` records the source MAC of the received Ethernet advertisement and its receiving interface. That neighbor is the next hop for routes learned from the advertisement. CP4 and CP5 therefore learn both the route and the link-layer address needed to use it.

`sendIPPacket()` copies the selected MAC and calls `sendFrame()` with EtherType `0x0800`. For a remote destination, this MAC belongs to the immediate neighbor, while the IPv4 destination remains the final host. Each forwarding router selects its own next-hop MAC and constructs a new Ethernet header.

The source also contains an ARP fallback for a route with an all-zero MAC: it searches the ARP cache for the route's next-hop IP and, on a miss, sends requests and retries. This branch requires a meaningful next-hop IP and an active receiver to process replies. The public manual-route API supplies a MAC, and DV learns a MAC directly; the recorded checkpoints use these two paths and do not depend on an ARP exchange. The ARP fallback is not described as the default path for these experiments.

PT3 - IPv4 sending, receiving, and forwarding

Sending

`sendIPPacket()` checks the payload pointer and length and the protocol range. Payloads above 1480 bytes are rejected, so the 20-byte header and payload fit in a 1500-byte Ethernet payload. The generated header has version 4, IHL 5, TOS 0, identification 0, DF set, fragment offset 0, and TTL 64. Multibyte fields use network byte order, and the IPv4 header checksum is calculated before transmission. A failed route lookup returns `ENETUNREACH`.

Receiving and local delivery

The frame dispatcher accepts IPv4 unicast frames addressed to the receiving interface and ignores frames whose source is that interface's own MAC. This prevents libpcap from feeding an outgoing frame back into the forwarding path.

`handleIPPacket()` checks version, header length, total length, fragmentation, and header checksum. It uses the IP total length rather than the captured Ethernet payload length, thereby excluding Ethernet padding from local delivery. A destination matching **any registered interface's cached IPv4 address** is local. The registered IP callback receives the complete IP datagram, with borrowed buffer ownership.

Forwarding

A nonlocal unicast datagram needs a valid route and TTL greater than 1. The handler copies the datagram, decrements TTL, recomputes the header checksum, and sends it through the selected interface and next-hop MAC. The source and destination IP addresses and payload are preserved. Fragments, malformed IPv4 datagrams, TTL exhaustion, and missing routes are dropped. Normal IPv4 drops and forwarding-send failures return 0 to the frame receive loop so that listening continues.

WT4 - Distance-vector routing

Metric, initialization, and advertisement

The algorithm follows distance-vector/Bellman-Ford updates with unit link cost. A received metric m yields candidate cost $\min(m+1, 16)$. Metrics 0-15 are reachable; **16 means unreachable**. Each interface contributes its own IPv4 address as a local `/32` route with metric 0. This lets the experiments measure distances to host addresses without treating a shared link subnet as a remote host.

After registering all interfaces and initializing local routes, the program begins reception and advertises immediately. It subsequently calls `updateRoutingTable()` and `sendDV()` approximately once per second. The routing table has at most 64 entries.

The wire protocol is a **custom Ethernet DV protocol**, using EtherType `0x88B6`. It uses hop-count infinity and poison reverse, but is not a RIPv1/RIPv2 wire-format implementation.

DV payload offset	Size	Meaning
0	1 byte	Version, currently 1.
1	1 byte	Reserved, must be zero.
2-3	2 bytes	Entry count, network byte order.
Each entry +0	4 bytes	Destination prefix.
Each entry +4	4 bytes	Subnet mask.
Each entry +8	4 bytes	Advertised metric, network byte order.

The header is 4 bytes and each entry is 12 bytes. A full 64-entry advertisement occupies 772 bytes, within the Ethernet payload limit. Advertisements use the Ethernet broadcast address on each interface.

Processing an advertisement

Before changing routes, `handleDV()` validates the complete message: source MAC, version, reserved byte, count/length bounds, contiguous masks, canonical prefixes, and metric range. Ethernet padding is permitted after the declared entries.

Routes are identified by **destination prefix and mask**. The receiving interface together with the Ethernet source MAC identifies the neighbor. A new reachable prefix is inserted if capacity permits. For an existing dynamic route:

- An update from the current next hop is accepted even if the metric becomes worse or unreachable.
- A different neighbor replaces the current route when its reachable path is shorter, or when the current route is invalid.
- Equal-cost advertisements from another neighbor retain the current next hop.
- Local and manually configured routes are protected from dynamic replacement.

Only the selected path is stored. The implementation does not retain every neighbor's alternative vector. Prefix length determines forwarding selection; hop count compares alternatives to the **same** prefix.

Poison reverse, timeout, and cleanup

For each outgoing interface, `sendDV()` takes a synchronized table snapshot. A dynamic route learned through that interface is advertised back with metric 16. Local/manual routes are advertised normally. Poison reverse changes the outgoing advertisement, not the stored route.

A dynamic route expires when no accepted reachable update has refreshed it for 5 seconds. It becomes invalid with metric 16 and remains available for withdrawal advertisements for another 5 seconds before deletion. Repeated unreachable updates do not restart this garbage-collection timer. A suitable reachable update can restore an invalid route. Lookup skips invalid/infinite routes immediately, without waiting for deletion.

Concurrency and boundary conditions

Reentrancy in the running stack

Each registered interface has separate RX and TX libpcap handles. One receive thread owns each RX handle. `sendFrame()` serializes injection and associated TX error access with a mutex, allowing periodic DV transmission and receive-triggered forwarding to share the sending path.

The routing table and ARP cache each have a mutex. Route updates, timeout/cleanup, and lookup execute under the route mutex. `findRouter()` returns a **thread-local copy**, so a caller does not retain a pointer into a table that another thread may move during garbage collection. The copy remains usable until that thread's next lookup. DV sending copies a snapshot under the mutex and releases the lock before frame transmission.

Interface registration and callback installation occur before receive threads start. The registry is then read-only. Cleanup is performed after its users have stopped, or by terminating the standalone experiment process. Concurrent registry reconfiguration or callback replacement is outside this operating contract. Application callbacks can be invoked by different interface threads and must synchronize their own shared application state.

Conditions addressed in the implementation

Condition	Handling
Ethernet padding after a short IP packet	IP total length bounds the delivered or forwarded datagram.
Captured outgoing IPv4 frame	Dispatcher rejects an interface's own source MAC and frames not addressed to its MAC.
Destination is another local interface	Local-address detection checks all registered interfaces.
Bad IPv4 version, lengths, or checksum	Drop the datagram and continue listening.
Fragmented IPv4 packet	Drop, as expressly permitted by the handout.
TTL at most 1 or no forwarding route	Drop without terminating the receive loop.
Malformed DV entry late in a message	Validate all entries before any route mutation.
Current next hop advertises a worse metric	Accept it, allowing loss of reachability to propagate.
Equal-cost alternative	Keep the selected neighbor to avoid unnecessary switching.
Repeated poison advertisement	Preserve the first invalidation time, allowing eventual cleanup.
Routing table at capacity	Refuse new entries; existing routes can still be updated.
Interface temporarily down	Log send failures; periodic processing continues. Recovery is exercised in CP4/CP5.

The implementation uses a maximum of 16 registered devices and 64 routes, and caches the first IPv4 address of each interface at registration. Interface addresses are configured before registration in all experiments. The ARP cache contains up to 128 IP-to-MAC mappings and is protected independently from the route table.

Poison reverse reduces loops, but it does not guarantee instantaneous convergence or eliminate every multi-router count-to-infinity scenario. Periodic updates, finite infinity, and route expiry determine eventual behavior. The checkpoint results establish the recorded stable states and actual deliveries, rather than a universal convergence-time bound.

All mandatory Lab 2 tasks are covered. Sending uses default TOS and no IP options; received options are not interpreted. Dropping fragments and omitting TOS/options support follow Section 3.3.1 of the handout. Accordingly, `not-implemented.pdf` is an empty file, as instructed for a completed lab.

CP3 - IPv4 construction and packet bytes

Evidence: `checkpoints/cp3/capture.typescript`, `send.typescript`, and `ipv4.pcap`.

The topology is a direct link between `cp3ns1` (`10.33.1.1/24`) and `cp3ns2` (`10.33.1.2/24`). The sender installs a `/32` manual route using the peer's actual MAC and calls `sendIPPacket()` with protocol 253 and the eight-byte string `lab2-cp3`. This experiment verifies IPv4 construction and transmission; dynamic multi-hop routing is checked separately below.

The complete captured Ethernet frame is:

```
0x0000: 7675 a67f 1b5f 4227 0d8e e9dd 0800 4500
0x0010: 001c 0000 4000 40fd 23a1 0a21 0101 0a21
0x0020: 0102 6c61 6232 2d63 7033 0000 0000 0000
0x0030: 0000 0000 0000 0000 0000 0000
```

Offsets below are **decimal and zero-based from the Ethernet frame start**. The IPv4 datagram starts at frame offset 14; its first byte is `0x45`.

Frame bytes	Value	Interpretation
0-5	`76 75 a6 7f 1b 5f`	Destination MAC: `76:75:a6:7f:1b:5f`.
6-11	`42 27 0d 8e e9 dd`	Source MAC: `42:27:0d:8e:e9:dd`.
12-13	`08 00`	EtherType `0x0800`: IPv4.
14	`45`	High 4 bits `0100`: version 4. Low 4 bits `0101`: IHL 5, or 20 bytes.
15	`00`	DSCP and ECN are zero.
16-17	`00 1c`	IPv4 total length: 28 bytes.
18-19	`00 00`	Identification: 0.
20-21	`40 00`	Flags `010`: reserved 0, DF 1, MF 0. The 13-bit fragment offset is 0.
22	`40`	TTL: 64.
23	`fd`	Protocol: 253. The test payload has no TCP/UDP header.
24-25	`23 a1`	IPv4 header checksum: `0x23a1`.
26-29	`0a 21 01 01`	Source address: `10.33.1.1`.
30-33	`0a 21 01 02`	Destination address: `10.33.1.2`.
34-41	`6c 61 62 32 2d 63 70 33`	ASCII `l a b 2 - c p 3`, with no trailing NUL.
42-59	Eighteen `00` bytes	Ethernet padding, outside the IPv4 datagram.

The captured frame has $14 + 28 + 18 = 60$ bytes, excluding FCS. The IP length is $20 + 8 = 28$ bytes. Independently summing the 16-bit header words with end-around carry, including the checksum, yields `0xffff`, confirming a valid header checksum. Both terminal recordings ended normally.

To inspect the original bytes from the repository root:

```
sudo tcpdump -nn -e -vvv -XX -r checkpoints/cp3/ipv4.pcap
```

CP4 - Four-node discovery, failure, and recovery

Evidence directory: `checkpoints/cp4/`.

Recordings: `control.typescript`, `ns1.typescript` through `ns4.typescript`. **Capture:** `forward.pcap`.

```
ns1 -------- ns2 -------- ns3 -------- ns4
       10.34.1/24   10.34.2/24   10.34.3/24
```

Link	Left interface address	Right interface address
ns1-ns2	`10.34.1.1/24`	`10.34.1.2/24`
ns2-ns3	`10.34.2.1/24`	`10.34.2.2/24`
ns3-ns4	`10.34.3.1/24`	`10.34.3.2/24`

Each namespace runs one checkpoint router with all its interfaces. Kernel IPv4 forwarding is disabled, and `bypassKernel` installs INPUT, OUTPUT, and FORWARD DROP rules. Packet transmission and forwarding therefore use the userspace implementation. No manual remote routes are added.

The test queries ns1's route to `10.34.3.2` and ns4's route to `10.34.1.1`. The query runs **inside the same process** as the routing protocol, since its route table is process-local.

Phase	ns1 to ns4	ns4 to ns1	Data delivery
Initial convergence	Metric 3, learned from DV	Metric 3, learned from DV	`cp4-before` arrives, TTL 62.
Both ns2 interfaces down	`UNREACHABLE`	`UNREACHABLE`	Send command reports no valid route.
Both interfaces restored	Metric 3, learned from DV	Metric 3, learned from DV	`cp4-after` arrives, TTL 62.

The selected ns1 outlet is `cp4veth1-2`, with next-hop MAC `d2:53:38:07:e9:55`, the adjacent ns2 interface. The actual ns4 receive records include:

```
RX src=10.34.1.1 dst=10.34.3.2 ttl=62 payload=cp4-before
RX src=10.34.1.1 dst=10.34.3.2 ttl=62 payload=cp4-after
```

For the failure phase, `ip link set ... down` disables both ns2 interfaces. Its sending errors are retained in the recording. The four processes remain running. After dynamic routes expire, both endpoints report unreachable. Bringing the interfaces back up allows those same processes to relearn the routes.

`forward.pcap` independently contains the two expected datagrams, with valid IPv4 header checksums. TTL starts at 64 and becomes 62 after forwarding at ns2 and ns3; the destination host does not decrement it for local delivery. All five terminal recordings ended normally.

These observations satisfy the three CP4 states: discovery, loss of discovery after isolating ns2, and recovery after reconnection. They do not measure exact convergence latency. An unreachable lookup establishes the absence of a usable route, rather than directly inspecting whether its invalid entry has already been garbage-collected.

CP5 - Six-node distance matrix and isolation

Evidence: `checkpoints/cp5/control.typescript`, `cp5ns1.typescript` through `cp5ns6.typescript`, and `ns1.pcap` through `ns6.pcap`.

```
ns1 --- ns2 --- ns3 --- ns4
         |       |
        ns5 --- ns6
```

The six links are 1-2, 2-3, 3-4, 2-5, 3-6, and 5-6, using `10.35.1.0/24` through `10.35.6.0/24` in that order. In particular, **ns6 connects to ns3**. Each router registers all its interfaces. As in CP4, kernel IP processing and forwarding are bypassed.

Each host is represented by one of its registered IPv4 addresses: ns1=`10.35.1.1`, ns2=`10.35.1.2`, ns3=`10.35.2.2`, ns4=`10.35.3.2`, ns5=`10.35.4.2`, and ns6=`10.35.5.2`. The measured entries below come from actual `findRouter()` queries. An independent breadth-first search on the topology supplies the expected hop counts. Three successive complete query rounds must agree before a phase is accepted.

Initial measured distances

Source / destination	ns1	ns2	ns3	ns4	ns5	ns6
ns1	0	1	2	3	2	3
ns2	1	0	1	2	1	2
ns3	2	1	0	1	2	1
ns4	3	2	1	0	3	2
ns5	2	1	2	3	0	1
ns6	3	2	1	2	1	0

Measured distances after isolating ns5

Both ns5 interfaces, `cp5veth5-2` and `cp5veth5-6`, are set down. No routing process is restarted. After convergence, the measured matrix is:

Source / destination	ns1	ns2	ns3	ns4	ns5	ns6
ns1	0	1	2	3	∞	3
ns2	1	0	1	2	∞	2
ns3	2	1	0	1	∞	1
ns4	3	2	1	0	∞	2
ns5	∞	∞	∞	∞	0	∞
ns6	3	2	1	2	∞	0

Here ∞ means no usable route, represented by protocol metric 16. The other five nodes remain connected; their shortest distances do not change because the ns2-ns3-ns6 path remains. Restoring both ns5 interfaces, with the same processes, restores the complete initial matrix. The three phases cover 108 measured distance entries.

Actual packet delivery

Each phase also sends a uniquely labeled test payload for every ordered pair of distinct hosts. There are 30 successful deliveries initially, 20 while ns5 is isolated, and 30 after restoration. The 10 ordered pairs involving isolated ns5 report unreachable without sending. Destination callbacks and six independent receive-side captures verify all **80 successful deliveries**, with no duplicate test payloads. Every captured header checksum is valid, and TTL equals $65 - d$ for a route of d links. All capture processes report zero kernel drops.

The first run stopped capture before one buffered packet was written. That diagnostic run remains in `tmp/cp5-first-run-evidence`. The final run waits for the capture files to contain all expected packets before stopping and passes all checks. This is a capture-completion correction; the protocol source was unchanged. The retained CP5 directory contains the complete passing run.

CP6 - Longest-prefix matching

Evidence: `checkpoints/cp6/control.typescript`, `out-a.pcap`, and `out-b.pcap`.

```
cp6ns2 -------- cp6ns1 -------- cp6ns3
  outlet A peer   sender         outlet B peer
```

The sender's outlet A is `cp6veth1-2` (`10.36.1.1/24`), connected to `10.36.1.2`. Outlet B is `cp6veth1-3` (`10.36.2.1/24`), connected to `10.36.2.2`. Both peer interfaces are captured independently. Kernel IP processing is bypassed.

`src/test/cp6/lpm.c` first confirms that an empty route table has no route for `203.0.113.7`. It then inserts four routes in the deliberately mixed order **/24, /0, /32, /16**. All four routes are present before the matching and transmission checks. Their next-hop MACs are read from the actual peer interfaces.

Destination	Selected prefix	Outlet	Payload
`10.60.1.42`	`10.60.1.42/32`	A	`cp6-host`
`10.60.1.43`	`10.60.1.0/24`	B	`cp6-subnet`
`10.60.2.42`	`10.60.0.0/16`	A	`cp6-network`
`203.0.113.7`	`0.0.0.0/0`	B	`cp6-default`

For each destination, the test compares the returned prefix, mask, interface, and next-hop MAC before calling the real `sendIPPacket()`. The `/32` case must override a matching `/24`, `/16`, and default route. The `/24` case overrides `/16` and default, `/16` overrides default, and the final case uses the default route. Because the insertion order differs from prefix-length order, these results establish longest matching prefix selection rather than first/last inserted selection.

The raw query/transmission output records:

```
PASS dst=10.60.1.42 prefix=10.60.1.42 mask=255.255.255.255
PASS dst=10.60.1.43 prefix=10.60.1.0 mask=255.255.255.0
PASS dst=10.60.2.42 prefix=10.60.0.0 mask=255.255.0.0
PASS dst=203.0.113.7 prefix=0.0.0.0 mask=0.0.0.0
```

The excerpt omits outlet/payload fields for compactness; the original lines are in `control.typescript`. Outlet A captures exactly `cp6-host` and `cp6-network`; outlet B captures exactly `cp6-subnet` and `cp6-default`. All four datagrams retain TTL 64 because this is a sending-path outlet test. Destination/source IPs, payloads, IPv4 header checksums, and the destination Ethernet MACs match the selected routes. Both capture processes exit normally with zero kernel drops.

Evidence and reproduction

Each checkpoint directory keeps only `README.md`, original `.typescript` recordings, and `.pcap` captures. Detailed commands and the per-checkpoint explanation are in those files. CP3/CP4 were recorded interactively; CP5/CP6 were executed by the supplied experiment runners. The CP5/CP6 runs stopped their processes and removed their own namespaces after verification.

Build with `make all cp3 cp4 cp5 cp6`. For an automatic rerun, use `src/test/cp5/run.py` or `src/test/cp6/run.py` with a new result-directory argument, under `script -q -f -e -c ...`, as shown in each checkpoint README. Auxiliary CSV/JSON data and capture diagnostics go to `tmp/`.

This report covers PT3, WT2-WT4, and CP3-CP6. The task definitions and permitted simplifications follow Sections 3.3 and 3.3.1 of the supplied assignment `README.pdf`; the report/evidence arrangement follows Section 6. Claims about execution refer to the preserved recordings and captured packets described above.